

BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE PILANI
K K BIRLA GOA CAMPUS

2024-25 Sem. I	Compre Regular	Open Book Paper Only
Principles of Programming Languages	CS F301	40%=80 marks; Pages=2; Questions=6
6 Dec 2024	180 min	1000-1300 hrs

Q1. For each kind of names described below, indicate its outermost scope for each of C, C++, Java. [Marks **16**]

Kinds of names: a) Formal parameters to a function. b) Local variables. c) Static variables. d) Labels. e) Type parameter in templates or generics (e.g. `T` in `ArrayList<T>`). f) An `enum` type defined inside a class. g) An `enum` type defined inside a class module (`.java`) but outside the class definition. h) Static members of a class.

The available scopes (with increasing coverage) are: 1 *expression*, 2 *block*, 3 *procedure/method*, 4 *class*, 5 *class hierarchy*, 6 *module*, 7 *package*, 8 *namespace*, 9 *program-global*, 10 *shell-global*. The last two mean: 9 global for the program and 10 global for the filesystem (e.g. JVM classpath).

The answers must be in the format “ $\alpha \leftrightarrow n$ ” where α is the a to h name kind specifier and n the scope number specifier above. Any ambiguity may lead to loss of marks and recheck claims.

Q2. Suppose you are developing a stock market transaction engine in C or C++ or Java. During calculations, before the final bottomline, no precision loss must happen, otherwise the cumulative error in transactions of billions can be an unaffordable several millions. The database is in typical spreadsheet format (₹ 999999999.99). Which of the following will work with no precision loss nor efficiency loss? Why? [Marks **15**]

- A wrapper class for all calculations with the whole and the fraction both represented by integers and actual computations in fixed point using integer arithmetic.
- A wrapper class for all calculations with the whole and the fraction both represented by long integers and actual computations in fixed point using long integer arithmetic.
- A long double type for all numerical calculations.
- The `double` type for all numerical calculations.
- The `float` type for all numerical calculations.

The answer must be in a 3-column table:

Number	Works?	Reason in no more than 2 sentences
x	Sometimes	Only when the reason is reasonable, precise and sound. Jargon-bubble fables from P. T. Chatji will entail penalty without pity.

Q3. Can you ensure that your developed class methods cannot be used except by instances of this class only, not even instances of its subclasses? Answer in the context of both C++ and Java. For a ‘yes’ show how; for a negative answer argue why not. Precision, brevity, clarity matter most. [Marks **10**]

Q4. You want to use a certain procedure that is a non-static method of a class, but you do not want to instantiate the class (or any of its subclasses). You have only its binary (compiled byte-code or object-code), no access to the source code and you cannot disassemble its binary code or otherwise reproduce its source code as a global function or a method of another, new class. Is this a dead-end? Or is there some way still to use the procedure without instantiating any class in its hierarchy? Answer with precise and sound justification for C++, Java, Python, each in no more than 2 sentences each. For each language, write the answer in a separate paragraph, suitably entitled, clearly demarcated from others. [Marks 12]

Q5. Can a program read a valid lambda expression at run-time from the user interactively, validate it, and then apply it (i.e. make a function call) to suitable actual parameters? Answer with precise and sound justification for C++, Java, Python, and any one of Lisp/ML/Scheme/CaML/OCaML/Ruby each in no more than 2 sentences each. For each language, write the answer in a separate paragraph, suitably entitled, clearly demarcated from others. [Marks 15]

Q6. Which of the languages (among C, C++, Java, Python, Lisp) allow the programmer to specialise each instance of a class (or a struct in C) in storage structure (data members' types, sizes, sequences, layouts), allocation, and methods (in C, function pointers as data members of structs)? Write for each language with clear, precise, brief reasoning in no more than 2 sentences each. For each language, write the answer in a separate paragraph, suitably entitled, clearly demarcated from others. [Marks 12]

SOLUTION

Answer 1. a) 3 b) 2 c) 2 d) 2 e) 4 f) C/C++: 6; Java: 5 g) C/C++: 6; Java: 7 h) 5.

Answer 2. The basic idea is to recognize that precision is not lost only when we do fixed point or integer calculations without a fixed bound on representation length: lengthening the representation as the required precision grows. Like Lisp: try running this in Lisp:

```
(defun power (m n)
  . (if (< n 0)
    . . (/ (power m (+ n 1)) m)
    . . (if (> n 0)
    . . . (* (power m (- n 1)) m)
    . . . 1
    . . )
    . )
)
(power (/ 2 3) 5)
(power (/ 2 3) -7)
```

and see if you get 32/243 and 2187/128 instead of 0.1317 and 17.086.

Number	Works?	Reason in no more than 2 sentences
i	Mostly	Except when multiplication and division make the magnitudes go above/below $2^{\pm 31} \approx 10^{pm9}$ – i.e. a few billions or their reciprocal.
ii	Always	Of course as long as magnitudes are within billion-billions or their reciprocals.
iii,iv,v	Mostly not	Because even if you divide a number by something like 7 in any floating point format, you lose some precision: 1/7 becomes 0.1428571428571428 4921269268124888185412 when it should have 142857 as the recurring fraction. How will you calculate interest and discounts in percentages and divide them in equal installments?

Answer 3. Yes, of course the method must be made private. However, there are caveats: if this private method overrides a parent class public method then via the parent class (pointer with a typecast in C++, or passing this class instance for a superclass parameter in C++/Java) this method can be accessed. Again, this is still usage by instances of *this* class only. So in C++, you can make a subclass that doesn't override this method, then calls the super-super class method by the pointer cast or parameter passing trick. See the given example program. Thus this solution (making a method private to restrict its usage to its instances only) is possible only when we are sure there are no base (super) classes with the same method public.

On the other hand Java doesn't allow this weakening of the access: private methods

```

1 using namespace std;
2 #include<iostream>
3 class one {
4     public: void Tell(void) { cout << "Here-1" << endl; }
5 };
6 class two: public one {
7     private: void Tell(void) { cout << "Here-2" << endl; }
8 };
9 class three: public two {
10 };
11 int main(void) {
12     two Two;
13     one *One = &Two;
14     three Three;
15     One->Tell();
16     Three.one::Tell();
17     return 0;
18 }

```

A C++ example for Answer 3.

cannot override public methods in superclasses. In Java, making the methods private is the clear answer, but making it **final** is not: A final method is accessible by subclass instances, just that it cannot be overridden in the subclasses. But a private method in Java is final too.

Answer 4. If at all, this would be imaginable in C++ with a pointer typecast:

```

thatClass *p = (thatClass*)(NULL);
p->thatMethod(...);

```

However, from the beginning, Stroustrup deprecated this:

The effect of calling a nonstatic member function (§9.4) of a class X for something that is not an object of class X is undefined.

■ For example,

```
((X*)0)->f();
```

is not guaranteed to work. In particular, this will not work for virtual functions on any implementation since the information needed to find the appropriate virtual function for an object of type X is not found at location 0. Even for nonvirtual functions, one should expect this trick to fail eventually because specialized C++ implementations might assume something about the contents of objects even when calling nonvirtual functions. In particular, one might expect implementations instrumented for debugging, interpreters, and implementations supporting **dynamic** loading to be sensitive to "housekeeping" information placed in the objects by the compiler.

Naturally, this trick would work only if *f()* either checks its *this* pointer before accessing any members or never uses any member of the object for which it was called. In the latter case, the function should have been declared *static* (§9.4). □

And, today any C++ compiler will warn against a NULL pointer as **this**. Any type-safety setting or advice to the compiler will entail treating this as an error. However, it may work when indeed no members of the class are to be accessed. Like, add the line

```
((one*)NULL)->Tell();
```

just before the main functions return in the C++ program given for Answer 3, and check the outcome.

For all the other languages, there is no way, there are no pointers to cast.

Answer 5. This is possible in functional languages only, wherein subroutines are truly first class citizens. Here is a Python example (interactive session transcript):

```
>>> eval(input("lambdaexp? -")+ ' (' +input("parameter? -")+ ' ) ' )
lambdaexp? (lambda x:x*x)
parameter? 5
25
>>>
```

C/C++/Java of course can't have this. Lisp/ML/Scheme/CaML/OCaML/Ruby of course allow this in ways similar to Python.

Answer 6. Only functional languages, i.e., among the ones mentioned in the question, Lisp and Python, (can and do) allow instance level specialization. Lisp examples will be like the Python examples below:

```
>>> class bird:
...     def __init__(self, wings):
...         self.wings = wings
...         if(wings>2):
...             self.tail = 1
...
>>> b1=bird(2)
>>> b2=bird(3)
>>> b2.tail
1
>>> b1.wings
2
>>> b2.wings
3
>>> b2.tail
1
>>> b1.tail
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: 'bird' object has no attribute 'tail'
>>>
```